

1. Backpropagation Algorithm in a Multilayer Perceptron

Definition

Backpropagation is a supervised learning algorithm used to train a **Multilayer Perceptron (MLP)**. It updates the weights of the network by minimizing the error between the actual output and the predicted output using **Gradient Descent**.

Step 1: Forward Pass

Input values are passed through the network.

Example:

- Input: Age = 45, Blood Pressure = High
- Hidden layer computes weighted sum.
- Activation Function (Sigmoid):

$$f(x) = \frac{1}{1 + e^{-x}}$$

Suppose the network predicts

Predicted Output = 0.75

Actual Output = 1 (Yes)

Step 2: Error Calculation

Error is calculated as

$$\begin{aligned} \text{Error} &= \text{Actual} - \text{Predicted} \\ \text{Error} &= 1 - 0.75 = 0.25 \end{aligned}$$

Loss Function (Mean Squared Error)

$$\begin{aligned} \text{MSE} &= \frac{1}{2} (\text{Actual} - \text{Predicted})^2 \\ &= \frac{1}{2} (0.25)^2 = 0.03125 \end{aligned}$$

Step 3: Backward Propagation

The error is propagated backward.

Weight update formula

$$\text{New Weight} = \text{Old Weight} + \eta \times \text{Error} \times \text{Input}$$

Assume

- Old Weight = 0.60
- Learning Rate = 0.10
- Error = 0.25
- Input = 1

$$\text{New Weight} = 0.60 + (0.10 \times 0.25 \times 1) = 0.625$$

The updated weight reduces the prediction error.

Influence of Learning Rate

- Small Learning Rate (0.01)
 - Slow learning
 - Stable convergence
- Large Learning Rate (0.9)
 - Faster learning
 - May overshoot the minimum and fail to converge.

Influence of Activation Function

- **Sigmoid**
 - Output between 0 and 1
 - Suitable for binary classification.
- **ReLU**
 - Faster convergence
 - Avoids vanishing gradient.
- **Tanh**
 - Output between -1 and 1
 - Better than sigmoid in some hidden layers.

Conclusion

Backpropagation repeatedly performs **Forward Pass** → **Error Calculation** → **Backward Pass** → **Weight Update** until the prediction error becomes very small, improving classification accuracy.

2. Genetic Algorithm

Dataset (Chosen Example)

Candidate Speed Accuracy Fitness

A	70	80	150
B	90	60	150
C	80	90	170
D	60	70	130

$$\text{Fitness} = \text{Speed} + \text{Accuracy}$$

Generation 1

Initial Population

Candidate Fitness

A	150
B	150
C	170
D	130

Best Candidate = C

Step 1: Selection

Choose candidates with highest fitness.

Selected

- C
- A

Step 2: Crossover

Parent 1

Speed = 80

Accuracy = 90

Parent 2

Speed = 70

Accuracy = 80

After crossover

Child

Speed = 80

Accuracy = 80

Fitness

$$80 + 80 = 160$$

Step 3: Mutation

Increase Accuracy slightly.

Child

Speed = 80

Accuracy = 95

Fitness

$$80 + 95 = 175$$

Generation 2

Candidate	Fitness
------------------	----------------

C	170
---	-----

Mutated Child 175

Best Individual

Speed = 80

Accuracy = 95

Fitness = 175

Genetic Algorithm Operations

Selection

Chooses individuals with the highest fitness values so better solutions survive.

Crossover

Combines good characteristics of two parents to produce improved offspring.

Mutation

Randomly changes genes to introduce diversity and avoid local optimum.

Final Optimized Solution

Speed Accuracy Fitness

80 95 175

This is the optimal solution after evolution.

3. Neural Network Classification Using Backpropagation

Dataset

Example

	Age	Blood Pressure	Glucose	Lifestyle Risk	Actual	Predicted
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

	Age	Blood Pressure	Glucose	Lifestyle Risk	Actual	Predicted
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

Training using Backpropagation

The neural network receives the input features:

- Age
- Blood Pressure
- Glucose Level
- Lifestyle Risk

These values pass through hidden neurons (Forward Pass).

The predicted output is compared with the actual output.

The prediction error is calculated.

Backpropagation updates the weights to reduce the error.

The process repeats until the error becomes very small.

Confusion Matrix

Treat **Yes** = **Positive** and **No** = **Negative**

Example 1

Actual = Yes

Predicted = Yes

True Positive (TP)

Example 2

Actual = Yes

Predicted = No

False Negative (FN)

Example 3

Actual = No

Predicted = No

True Negative (TN)

Example 4

Actual = Yes

Predicted = Yes

True Positive (TP)

Example 5

Actual = No

Predicted = Yes

False Positive (FP)

Final Values

Measure	Value
True Positive (TP)	2
True Negative (TN)	1
False Positive (FP)	1
False Negative (FN)	1

Performance Metrics

Accuracy

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ &= \frac{2 + 1}{5} = \frac{3}{5} = 0.60 \end{aligned}$$

Accuracy = 60%

Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{2}{2 + 1} = \frac{2}{3} = 0.667$$

Precision = 66.7%

Recall

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{2}{2 + 1} = \frac{2}{3} = 0.667$$

Recall = 66.7%

F1-Score

$$\begin{aligned} F1 &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \\ &= \frac{2 \times 0.667 \times 0.667}{0.667 + 0.667} = 0.667 \end{aligned}$$

F1-Score = 66.7%

Performance Analysis

- The model correctly classified **3 out of 5** samples.
- Accuracy is **60%**, indicating moderate performance.
- Precision and Recall are **66.7%**, showing the model identifies positive cases reasonably well but still makes one false positive and one false negative.
- More training data and better feature selection can further improve the model.

Importance of Preprocessing

- Handle missing values.
- Normalize numerical features (Age).
- Encode categorical values (Blood Pressure, Glucose Level, Lifestyle Risk).
- Remove noisy or duplicate data.

These steps improve model stability and accuracy.

Importance of Feature Selection

Selecting only relevant features (Age, Blood Pressure, Glucose Level, Lifestyle Risk) helps to:

- Reduce overfitting.
- Improve prediction accuracy.
- Reduce training time.
- Increase model reliability.

Overall Conclusion: Backpropagation enables the neural network to learn from errors by updating weights iteratively. Using the given dataset, the confusion matrix values are **TP = 2**, **TN = 1**, **FP = 1**, **FN = 1**, giving an **Accuracy of 60%**, **Precision of 66.7%**, **Recall of 66.7%**, and **F1-Score of 66.7%**.